

# CP Generator

Generation, Diagnostics, and Animation of Origami Crease Patterns

---

Allan Pan

May 24, 2026

# What this project is

A desktop app that generates, diagnoses, and animates origami crease patterns on a square sheet — entirely from scratch.

`src/cp_generator/core.py` ·

`src/cp_generator/app.py` ·

`src/cp_generator/fold_sim.py`

## Five linked algorithms

1. Build a planar candidate graph.
2. Repair parity & optimize for Kawasaki.
3. Assign mountains/valleys.
4. Extract faces & animate the fold.
5. Diagnose what is only local, what is globally consistent, and what is still heuristic.

## Key objects

- Sheet  $[0, s]^2$ ; graph  $G = (V, E)$ , positions  $X_v \in \mathbb{R}^2$ .
- Partial crease labeling  $\tau : E \rightarrow \{U, M, V\}$ .
- Complete MV assignment  $\sigma : E \rightarrow \{-1, +1\}$ , with  $+1 = M$  and  $-1 = V$ .
- **Sector angle**: angle between consecutive creases at a vertex.
- **Half-edge**: oriented edge — determines which face lies to its left.

# Mathematical model of the program state

## Current object

$$\mathcal{C} = (G, X, \tau), \quad G = (V, E), \\ X : V \rightarrow [0, s]^2, \quad \tau : E \rightarrow \{U, M, V\}.$$

$$V_{\text{int}} = \{v \in V : X_v \notin \partial[0, s]^2\}.$$

At each  $v \in V_{\text{int}}$ , the incident creases are listed in cyclic order, with sector angles

$$\alpha_1(v), \dots, \alpha_{2m(v)}(v), \quad \sum_i \alpha_i(v) = 2\pi.$$

## Two vertexwise quantities

$$\kappa(v) = \left| \sum_{i \text{ odd}} \alpha_i(v) - \pi \right|$$

measures Kawasaki error, while

$$\mu_\tau(v) = M_\tau(v) - V_\tau(v)$$

measures Maekawa balance after MV assignment.

The pipeline constructs  $G$ , improves  $X$ , searches for  $\tau$ , then certifies as much global geometry as the code can justify.

# Algorithm I — Planar candidate graph

**Goal.** A planar graph dense enough for interesting folds, regular enough for stable local angle computation.

## Construction

1. Sample interior points; add four corners.
2. Snap near-boundary points to the boundary.
3. Compute  $G_0 = \text{Del}(\tilde{P})$ .
4. Delete edges lying entirely on one boundary side.
5. Use the remaining straight-line edges as  $E$ .

## Mathematical role

Delaunay does not prove foldability. It supplies a planar candidate graph with relatively regular local geometry, so the cyclic order and sector angles used later are numerically stable.

## Algorithm II — Parity repair & Kawasaki optimization

### Necessary condition

Every interior vertex must have **even degree**:  
 $d(v) \equiv 0 \pmod{2}$ .

**Repair.** Pair odd-degree vertices and delete a shortest path between each pair. Only the endpoint parities flip. (Heuristic — not globally optimal.)

**Geometry repair.** Let  $D(X) = \sum_v w_v \|X_v - X_{0,v}\|^2$ . Move vertices as little as possible, subject to the Kawasaki constraints:

$$\min_X D(X) + \frac{\alpha}{2} D(X)^2,$$

with one alternating-angle equality per vertex, since the other follows from  $\sum_i \alpha_i = 2\pi$ . Large boundary weights and small interior weights keep the square nearly fixed. Solved with SLSQP.

### Kawasaki's theorem (local) [Hul94]

Sector angles  $\alpha_1, \dots, \alpha_{2m}$  must satisfy

$$\alpha_1 + \alpha_3 + \dots = \alpha_2 + \alpha_4 + \dots = \pi.$$

## Algorithm III — Local constraints: Maekawa & Bern–Hayes

### Maekawa's theorem (local) [Hul94]

At every interior vertex,

$$M(v) - V(v) = \pm 2.$$

Used as an **admissibility filter**: reject any assignment violating this anywhere.

### Bern–Hayes-style reduction [BH96]

For each vertex, repeatedly **remove the smallest sector**: its two bounding creases are forced *equal* or *opposite* in sign.

1. Sort creases around the vertex by angle.
2. Find the smallest wedge; record its forced sign relation.
3. Recurse until the vertex is fully resolved.

Boundary vertices get temporary virtual rays to close the cyclic order.

## Algorithm III — Merging, exact search, & interlacing

Each vertex reduction yields sign chains:

$$\sigma(e_i) = \sigma(e_j) \quad \text{or} \quad \sigma(e_i) = -\sigma(e_j).$$

### Exact search

- Merge chains across shared creases into  $g$  free groups.
- Enumerate all  $2^g$  seed assignments; propagate implied signs.
- Keep only assignments with  $|M(v) - V(v)| = 2$  at every interior vertex.

Undetermined creases keep label  $-1$ ; no sign is invented.

### Interlacing preference

Among admissible assignments, prefer  $M, V, M, V, \dots$  over  $M, M, V, V, \dots$ , weight sharper wedges more, and subtract a large penalty for the obtuse same–different–same defects on the next slide.

Does not affect correctness.

## Algorithm IIIb — Repairing obtuse-angle glitches

### Glitch pattern

For consecutive creases  $e_i, e_{i+1}, e_{i+2}$  with consecutive sector angles  $\beta_i, \beta_{i+1}$ ,

$$\beta_i, \beta_{i+1} > \frac{\pi}{2}, \quad \sigma(e_i) = \sigma(e_{i+2}) \neq \sigma(e_{i+1}),$$

the code flags a local “same–different–same” obstruction across two consecutive obtuse wedges.

### Maekawa-preserving fix

Test exactly two monochromizing moves:

$$\sigma(e_{i+1}) \leftarrow \sigma(e_i) \quad \text{or} \quad \sigma(e_i), \sigma(e_{i+2}) \leftarrow \sigma(e_{i+1}).$$

Keep only a move that still satisfies

$$|M(v) - V(v)| = 2$$

at every interior vertex, then iterate while the total glitch count drops.

This is a targeted repair for a local/global mismatch: Maekawa alone may pass even though the resulting flat stack is visually and combinatorially implausible.

# Algorithm IV — Faces, reflection & 3D animation

## Face extraction

Add the square boundary, order neighbors by polar angle, and trace half-edges by the left-face rule

$$(u, v) \mapsto (v, w),$$

where  $w$  is the predecessor of  $u$  in the cyclic order at  $v$ . Each bounded counterclockwise cycle defines one face.

## Reflection rule

If two faces share crease  $\ell$ , the child is the parent reflected in  $\ell$ . Propagated from a root face outward; if any cycle is inconsistent, the method falls back to per-triangle propagation along a spanning tree.

## Kinematic animation [AE+15]

Around crease axis  $(p, q)$ , with sign  $\sigma(e) \in \{\pm 1\}$  and fold angle  $\theta$ :

$$T_{\text{child}} = R_{(p,q)}(\sigma(e)\theta) T_{\text{parent}}.$$

Rigid plates connected by hinge creases: no elasticity, no collisions. Triangulation is only for rendering, and small normal offsets keep layers visible near full closure.

# Algorithm IVb — Exact preview motion families

## Legacy layered

Render the exact tree pose and blend late into a separated flat stack:

$$P_f^{\text{legacy}}(t) = (1 - s(t))P_f^{\text{tree}}(t) + s(t)L_f.$$

## Balanced stack

Close seams by averaging shared vertices and rigidly refitting each face, then blend to a thinner final stack:

$$P_f^{\text{bal}}(t) = (1 - s(t))\mathcal{R}(P^{\text{tree}}(t))_f + s(t)B_f.$$

## Seamless rigid

At sample times  $t_j$ , solve per-face rigid transforms by

$$\min \lambda_s E_{\text{seam}} + \lambda_g E_{\text{guide}} + \lambda_h E_{\text{hinge}},$$

so seams close, faces stay near the exact guide pose, and neighboring faces track the guide hinge rotation.

Accepted samples are interpolated by SLERP. If one sample fails, hold the nearest solved exact sample instead of jumping to the older gappy fallback. At  $t = 1$ , render the exact folded figure.

If exact face reconstruction itself fails, the app still drops to the guarded triangle-mesh preview.

## Algorithm V — Diagnostic status map

Let  $\mathcal{C} = (G, X, \tau)$  be the current crease pattern, let  $V_{\text{int}} \subseteq V$  denote the interior vertices, and let

$$\kappa(v) = \left| \sum_{i \text{ odd}} \alpha_i(v) - \pi \right|$$

be the Kawasaki residual at  $v$ , where  $\alpha_1(v), \dots, \alpha_{2m(v)}(v)$  are the sector angles in cyclic order.

### Reported statuses

1. **Local status = pass** iff every  $v \in V_{\text{int}}$  has even degree and  $\kappa(v) \leq \varepsilon$ .
2. **Assignment status = pass** iff  $\tau$  is complete and  $|M_\sigma(v) - V_\sigma(v)| = 2$  for every  $v \in V_{\text{int}}$ .
3. **Global status = pass** iff the current embedding has no nonincident crease crossings, every crease borders exactly two traced faces, and reflection propagation is cycle-consistent on the face-adjacency graph.
4. **Exact-preview status = pass** iff the current geometry admits exact face reconstruction with no provisional signs and no mesh fallback.

### Non-certifying outcomes

A warning is reported whenever exact reconstruction needs a reference-geometry reset, provisional completion of  $\tau$ , or mesh fallback.

# What each pass actually certifies

## Local and assignment passes

$$\forall v \in V_{\text{int}}, \quad d(v) \in 2\mathbb{Z}, \quad \kappa(v) \leq \varepsilon,$$

and, after a complete assignment,

$$\forall v \in V_{\text{int}}, \quad |M_{\sigma}(v) - V_{\sigma}(v)| = 2.$$

This is a certificate of *vertexwise admissibility*, not of a global flat fold.

## Global and exact-preview passes

These certify properties of the *current embedding*: noncrossing creases, closed traced faces, two-sided face incidence on each fold, and cycle-consistent reflection propagation, with no provisional MV completion.

The deck therefore separates “locally admissible” from “globally certified current geometry” as distinct mathematical statements.

# Algorithm VI — Implemented global-consistency test

The current code does *not* solve arbitrary global flat-foldability. It implements the following certifier on the current embedding  $X$ :

## Implemented decision procedure

1. If nonincident fold segments cross, return `fail`. If there are no folds, return `not_run`. If no MV search has been attempted, return `unknown`.
2. Augment the fold graph by the square boundary, and order neighbors at each vertex by polar angle.
3. Trace bounded faces by  $(u, v) \mapsto (v, w)$ , with  $w$  the predecessor of  $u$  in the cyclic order at  $v$ . Reject repetition or nonclosure.
4. Require every fold edge to border exactly two traced faces. Build the face graph, choose a maximal-area root face, and set  $T_{f_0} = I$ .
5. Across each crease  $\ell$ , propagate  $T_g = R_\ell T_f$ . Hard cycle drift gives `fail`; soft drift gives `warning`.
6. Resolve fold signs from  $\tau$ . Missing signs are provisionally filled by face-depth parity, which downgrades to `warning`.

## Output interpretation

A final pass means: no crossings, successful exact face tracing, two-sided incidence on every fold, cycle-consistent reflections within tolerance, and no provisional MV completion.

# Scope of the local certificates

## Theorem 1 (Hull's local flat-foldability conditions)

*If a single-vertex crease pattern flat-folds, then its degree is even, its sector angles satisfy Kawasaki's alternating-sum identity, and every complete MV assignment satisfies Maekawa's relation  $|M - V| = 2$ . [Hul94]*

## Why this distinction matters

The implemented *local status* and *assignment status* test these vertexwise conditions up to numerical tolerance  $\epsilon$ . Bern–Hayes only derives local sign constraints and shrinks the search over  $\sigma$ ; it does not decide global flat-foldability. The deck therefore keeps “locally admissible assignment” and “globally certified current geometry” separate. [BH96]

# Meaning of a global or exact-preview pass

## Theorem 2 (Implemented exact-current-geometry certificate)

*If the algorithm on the previous slide returns global status = pass and exact-preview status = pass, then the current embedding of  $\mathcal{C}$  has a noncrossing augmented planar graph, a closed family of bounded traced faces, a two-face incidence relation on every fold edge, and a cycle-consistent family of flat reflections  $T_g = R_\ell T_f$ , all without provisional completion of the MV data.*

## Why this theorem matches the code

1. The core checker first rejects nonincident segment crossings.
2. The fold simulator traces bounded faces by cyclic half-edge traversal and aborts on repetition or nonclosure.
3. The exact solver requires two incident faces per fold and propagates transforms by explicit crease reflections.
4. Cycle drift beyond tolerance aborts; provisional sign completion downgrades pass to warning.

This is therefore a theorem about the *current geometry seen by the code*, not a complete decision procedure for arbitrary multi-vertex flat-foldability. [HZ25, Epp23]

# The whole mathematical picture

## Pipeline as one implication chain

sample points  $\implies$  planar candidate graph  $\implies$  even-degree repair  $\implies$  small Kawasaki residual  
 $\implies$  locally admissible MV constraints  $\implies$  exact search over sign groups  $\implies$  current assignment  $\sigma$   
 $\implies$  face extraction  $\implies$  reflection compatibility  $\implies$  exact or warning-level preview.

## Rigorous interpretation

Every arrow above is implemented in the code. The early arrows are geometric construction and local necessary conditions; the later arrows certify only the *current geometry*. No claim here is a complete decision procedure for arbitrary multi-vertex flat-foldability.

# Certified today vs. unresolved

## Currently certifiable

- Vertexwise even-degree and Kawasaki residual checks.
- Maekawa verification on a complete assignment  $\sigma$ .
- Explicit detection of nonincident crease crossings.
- Exact face reconstruction on the current geometry.

## Still non-certifying

- Parity repair is not globally optimal.
- Bern–Hayes reduction is local, not a full global classifier.
- Reference-geometry and mesh preview fallbacks remain heuristic.
- A true exact global checker is still needed for larger patterns.

## Best theoretical next step

The most plausible upgrade path is a bounded-instance exact checker based on ply and cell-adjacency complexity, built on the face arrangement already computed by the exact preview solver. [Epp23]

-  Marshall Bern and Barry Hayes.  
**The Complexity of Flat Origami.**  
SODA, 1996.
-  Thomas C. Hull.  
**On the Mathematics of Flat Origamis.**  
Congressus Numerantium, 1994.
-  Zachary Abel, Jason Cantarella, Erik D. Demaine, David Eppstein, Thomas C. Hull, Jason S. Ku, Robert J. Lang, and Tomohiro Tachi.  
**Rigid Origami Vertices: Conditions and Forcing Sets.**  
Journal of Computational Geometry, 2015.



David Eppstein.

**A Parameterized Algorithm for Flat Folding.**

arXiv:2306.11939, 2023.



Thomas C. Hull and Inna Zakharevich.

**Flat Origami Is Turing Complete.**

arXiv:2309.07932, 2025 version consulted.



Andreas Walker and Tino Stanković.

**Algorithmic Design of Origami Mechanisms and Tessellations.**

Communications Materials, 2022.